SWE5001 Practice Project · Graduate Certificate in Architecting Scalable Systems

Speaker Script and Live Demo Run Book

BlocBuy — Team 8 · Wednesday 21 October 2026, 16:15–16:55 SGT (Zoom)

Responsible	Lim Jiechao (presentation lead, #461); reviewer Liu Jin.
Companion files	`swe5001-presentation.html` — the deck (24 slides + backups BX, B1–B15; open in Chrome or Edge; **S** notes window with this script and a timer, **T** start timer, **O** overview, **F** fullscreen, **B** backup index) · `swe5001-presentation.pdf` — the same deck as a PDF (41 pages: 24 main, then backups).
Single source	`src/content.js` holds wording, timings, cues, Q&A and open items; `src/slides-main.js` and `src/slides-backup.js` hold the slide layouts and `src/script.js` this document; the UML diagrams are shared from `docs/presentation-kit/diagrams/`. Run `npm ci` once in `docs/presentation-kit/`, then `npm run all` in `docs/swe5001/presentation/` to rebuild everything in `dist/`.
Facts as of	18 Oct 2026. Evidence from 17 Oct; everything that depends on the 19 Oct session is marked and must be resolved Monday evening (§9).

How to use this document

- **Speakers:** read your segment (§1–§5) aloud against a timer at least twice. Each slide card shows the slot and the estimated speaking time of the sample wording at 130 words a minute. Cards marked **over by N s** need either a faster read or the listed trim. Rephrase in your own words; never read the slide.
- **Demo team:** §6 is the run book: preconditions, the Plan A/B decision, screen layout, then every step with what to operate, what to say, what you should see and when to switch to the fallback. §7 adds Plan C.
- **Lead:** the run sheet and coverage check below, the contingencies in §7, the day timeline in §8 and the open items in §9 are yours.

Markers

`[FILL 19 Oct]` value only exists after the Monday evidence session; do not present a guess · `[VERIFY]` claim unconfirmed or conflicting between sources; check the repo or run log · `[CONFIRM]` needs team agreement. They are visible on the slides on purpose and must all be gone after Tuesday's rehearsal.

Structure follows “04 Project Presentation Guidelines & Schedule” V2.8 exactly: one chapter per segment (1 Overview, 2 Project Conduct, 3 Logical, 4 Physical, 5 DevOps, 6 Demo), each opening with the guideline’s own sub-items.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:15	15 s	1	BlocBuy	Lim	1a	15 s
16:15:15	40 s	2	Group buying only works if enough people join in time	Lim	1a	37 s
16:15:55	40 s	3	A two-sided platform built around one interaction: forming a team	Lim	1b	36 s
16:16:35	25 s	4	Scope: the full buying journey, on three services	Lim	1c	**29 s**
16:18	45 s	5	Status: what is verified on AWS, and what is still open	Lim	2a	30 s
16:18:45	45 s	6	On schedule, about 50 person-days across five people	Lim	2b	36 s
16:20	90 s	7	Why three services, not five	Eric	3a	63 s
16:21:30	80 s	8	Four decisions shape the logical design	Eric	3a	57 s
16:22:50	90 s	9	Logical view: one direct call, everything else by events	Eric	3b	69 s
16:24:20	75 s	10	Bounded contexts decide the services; events are the contract	Eric	3c	51 s
16:25:35	85 s	11	How producers and consumers use the platform, and what reuse costs	Eric	3c	60 s
16:29	85 s	12	Physical decisions: managed, disposable, low cost	Liu Jin	4a	60 s
16:30:25	85 s	13	Physical view: private services behind CloudFront, across two zones	Liu Jin	4b	54 s
16:31:50	45 s	14	Each service scales on its own signal, and how far it goes	Liu Jin	4b	30 s
16:32:35	45 s	15	Each service owns its data	Liu Jin	4c	**50 s**
16:33:20	95 s	16	Team expiry, end to end: one transaction in, one transaction out	Yorik	4d	78 s
16:34:55	65 s	17	Failures are expected, and handled	Yorik	4d	42 s
16:38	65 s	18	One main branch; production changes only by promotion	Pan	5a	35 s
16:39:05	70 s	19	Every pull request is checked before it can merge	Pan	5b	47 s
16:40:15	95 s	20	One promotion builds, deploys, tests and releases, then destroys it all	Pan	5b	67 s
16:41:50	75 s	21	What the real cloud taught us	Pan	5c	39 s
16:43:05	55 s	22	No standing keys: who and what can touch AWS	Pan	5b	49 s
16:45	15 s	23	Live demo	Eric	6a	**16 s**
16:53	Q&A	24	Thank you	Lim	6b	

Segment talk budget vs slots vs estimated speaking time: §1 120 s / 120 s / 117 s · §2 90 s / 90 s / 66 s · §3 420 s / 420 s / 300 s · §4 420 s / 420 s / 314 s · §5 360 s / 360 s / 237 s · §6 480 s / 15 s / 16 s (§6 talk is the demo itself).

Timing check

3 slides read longer than their slot at 130 wpm: slide 4 (29 s / 25 s), slide 15 (50 s / 45 s), slide 23 (16 s / 15 s). Each card lists what to trim if the rehearsal runs long.

Q&A windows (inside the 40 minutes): 16:17–16:18 · 16:19:30–16:20 · 16:27–16:29 · 16:36–16:38 · 16:44–16:45 · 16:53–16:55. Background actions: **16:28** Yorik starts k6 V3 · **16:40** expiry-burst decision point (§6) · **16:44:30** screen-share switch Lim → Eric.

Every guideline item has slides and a speaker

Item	Guideline requirement	Slides	Speaker
1a	Explain the context and the business problem that the solution as a whole is trying to achieve.	1, 2	Lim
1b	Highlight the platform-thinking aspects of your solution.	3	Lim
1c	Explain the scope of the project.	4	Lim
2a	Explain the current status of the project including outstanding issues.	5	Lim
2b	Describe the actual project milestones and the rough total effort expended by each team member.	6	Lim
3a	Highlight the key architectural decisions that influence the logical architecture.	7, 8	Eric
3b	Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views.	9 (UML logical deployment view)	Eric
3c	Where relevant, highlight the elements of Domain Driven Design … how the platform is designed to interact with the producers and consumers.	10, 11	Eric
4a	Highlight the key architectural decisions that influence the physical architecture.	12	Liu Jin
4b	Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution.	13 (UML deployment), 14	Liu Jin
4c	Describe the persistence design of the solution.	15	Liu Jin
4d	Describe the detailed design of an architecturally representative user case or user story.	16 (UML sequence), 17	Yorik
5a	Describe the strategy to manage the source code and artifacts.	18	Pan
5b	Describe the DevOps pipeline and procedures.	19, 20, 22 (security controls)	Pan
5c	Highlight any technical findings and issues encountered.	21	Pan
6a	Demonstrate some of the significant use cases of the solution.	23 + demo step 1	Eric
6b	Demonstrate how the solution deals with some of the important quality attributes …	24 + demo steps 2–6	Lim

Every briefing question has an answer on a slide

Briefing	Where

Q1 pain points	2
Q2 platform, common services, reusable frameworks	3, 9, 11; B5, B10
Q3 benefit of building on the platform, demonstrated	11 (measured reuse)
Q4 scale and proof	14 (capacity statement), demo step 2; B12
Q5 demonstrate technical and non-technical benefits	12 (measured cost), 14, 19, demo
Q6 value of cloud native, demonstrated	12 (US$0.49 measured), 13, 20
Q7 SDLC automation reduces effort	19 (5,252 runs), 20
Q8 effort within guideline	6
Success criterion: seed, producers, consumers	3
Success criterion: more seeds in future	3 (future seeds line)
Success criterion: analytics	B11 (Q&A)
Module: robust API access; security controls	11, B10; 22

Cue words in the team backchannel (never the Zoom chat)

Message	Meaning	Sent by
`1 MIN`	Speaker has one minute of talk left in the segment	Lim
`TIME`	Stop talking; move to questions or the hand-over	Lim
`K6 GO / K6 UP`	Start the V3 load / load is running and healthy	Lim / Yorik
`SCALED n`	Commerce reached n running tasks (from the dashboard)	Eric
`PLAN A / PLAN B`	Burst decision at 16:40 (§6)	Liu Jin
`ID BACK`	identity-svc is running again after the outage drill	Yorik
`CLIP n`	Live step n failed; Liu Jin is ready to share the recording for step n	Eric → Liu Jin
`SHARE`	Next person: start screen sharing now	Lim

1 Overview (2+1 minutes)

Clock	16:15:00 – 16:18:00 (2 min talk + 1 min Q&A; talk ends 16:17:00)
Owner	Lim Jiechao
Slides	1, 2, 3, 4

Guideline requirement (V2.8)

1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Highlight the platform-thinking aspects of your solution.
1c — Explain the scope of the project.

Slide 1 BlocBuy

16:15:00–16:15:15 · 15 s

Speaker Lim · covers guideline item **1a**

Say

Good afternoon, we're Team 8, and this is BlocBuy, a group-buying platform built to scale on AWS. We'll go from the problem, to the architecture, to DevOps, and finish with a live demo.

≈ 33 words · ≈ 15 s at 130 wpm · slot 15 s

Cues

- Lim opens swe5001-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen) and T there at 16:15:00. In Zoom, share only the deck window.
- Liu Jin watches ECS health on the dashboard, muted.

Hand-over *"So, what problem are we solving?"*

Slide 2 Group buying only works if enough people join in time

16:15:15–16:15:55 · 40 s

Speaker Lim · covers guideline item **1a**

Say

Group buying gets people a lower price by buying together, but only if enough people join before the deadline. Today this runs on chat groups and spreadsheets. Buyers can't see how close a deal is, organisers collect money and refund people by hand, and small sellers have nowhere to post team prices. BlocBuy automates all of it: you pay to join a team, the price drops as members join, and if the team doesn't fill in time, everyone is refunded automatically.

≈ 81 words · ≈ 37 s at 130 wpm · slot 40 s

Hand-over *"And we built it as a platform, not a shop."*

Slide 3 A two-sided platform built around one interaction: forming a team

16:15:55–16:16:35 · 40 s

Speaker Lim · covers guideline item **1b**

Say *The core interaction is forming a team at a team price. Sellers publish offers; buyers browse and join. Buyers who start a team or share an invite bring new people in, so consumers also act as producers. Every new member makes the deal better for everyone, which is our network effect, and we seed it with a few curated offers. Identity, notifications and payments are shared services, so another group-buying business, say event tickets, could reuse them.*

≈ 77 words · ≈ 36 s at 130 wpm · slot 40 s

Cues

- The future seeds now sit on the slide (briefing success criterion "support for more seeds in future"); no need to list them aloud.

Hand-over *"Here's what we actually built."*

Slide 4 Scope: the full buying journey, on three services

16:16:35–16:17:00 · 25 s

Speaker Lim · covers guideline item **1c**

Say *In scope is the whole journey: sign in, browse, start or join a team, pay, and the team either succeeds or expires with refunds, plus sharing, order history and admin listings. It runs on three services, and we added event-driven notifications. Out of scope: multi-region, real money, and email or push. Splitting payment out is our documented next step, not a gap.*

≈ 62 words · ≈ 29 s at 130 wpm · slot 25 s · **over by 4 s: rehearse at pace or trim — If over time, drop "and we added event-driven notifications" (slide 5 covers it).**

Cues

- 16:17:00 — questions (1 minute). Lim repeats each question in one line and names who answers.

Hand-over (after questions) "Now, where the project stands."

Q&A (16:17–16:18)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
What makes this a platform rather than one application?	Two reusable platform services (identity, notification) that any seed can use; a new consumer is a queue plus a subscription; the starter library and golden-path module make a new service cheap. Details in segment 3.	Lim → Eric
Why build group buying at all — isn't this a solved problem?	Communities still coordinate in chat groups; the pain is coordination, payment collection and refunds, which the platform automates end to end.	Lim
What would a second seed need?	Its own catalog and rules in a new service from the golden path; identity, notification and settlement reused as they are. Future seeds are on slide 3.	Lim

2 Project Conduct (1.5+0.5 minutes)

Clock	16:18:00 – 16:20:00 (1.5 min talk + 0.5 min Q&A; talk ends 16:19:30)
Owner	Lim Jiechao
Slides	5, 6

Guideline requirement (V2.8)

2a — Explain the current status of the project including outstanding issues.
2b — Describe the actual project milestones and the rough total effort expended by each team member.

Slide 5 Status: what is verified on AWS, and what is still open

16:18:00–16:18:45 · 45 s

Speaker Lim · covers guideline item **2a**

Say

Here is exactly where we stand. On 17 October we built the full three-service environment on AWS from nothing, passed nine smoke checks through CloudFront, and tore down all 115 resources cleanly. On 19 October we ran `[FILL 19 Oct: read the green items]`*. Still open: an interrupted start-up can leave auto-scaling paused, which we repair manually; the database and cache run in one zone, a deliberate cost choice; and* `[FILL 19 Oct: any 19 Oct item that failed]`*.*

≈ 65 words · ≈ 30 s at 130 wpm · slot 45 s

Cues

- Rule for this slide: say only what docs/swe5001/evidence/ records. The assessor is likely to probe status claims in Q&A.
- Monday evening: green ticks only for items with evidence. Anything not run on 19 Oct moves to "Still open" with one honest line, e.g. "V4 queue scaling: verified locally (141 s end to end), not yet on AWS".

Hand-over *"And how we got here."*

Slide 6 On schedule, about 50 person-days across five people

16:18:45–16:19:30 · 45 s

Speaker Lim · covers guideline item **2b**

Say

We kept to the proposal's schedule: proposal on 4 October, first deployment to AWS two days later, the three-service split by the 14th, our load baseline on the 16th and the first green three-service release on the 17th, and the final release and test run on the 19th. In total it took about 50 person-days, which is what the course expects. We estimated each person's share from the issues they delivered, and you can see who led which part.

≈ 79 words · ≈ 36 s at 130 wpm · slot 45 s · if short of time: If over time, name only three milestone dates (4, 14 and 17 October).

Cues

- `[CONFIRM: effort split agreed by all five (open item 7)]`
- 16:19:30 — questions (30 seconds).

Hand-over (after questions) "Eric will take you through the logical architecture."

Watch out

Slide 5 must show the real state on 21 Oct. After the 19 Oct session, give a green tick only to items with evidence in `docs/swe5001/evidence/`; move everything else to “Still open”. Never claim a result that the evidence folder does not record.

Q&A (16:19:30–16:20)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How was the work split, and is effort within the guideline?	About 50 person-days in total, in line with the course guideline: between 8.5 and 12.5 per person. We took the planned hours of the issues each person delivered, credited each issue to whoever actually did it, and the team agreed the final split.	Lim
What slipped?	Name it plainly from slide 5, then the plan and date to close it. Full results table: backup B12.	Lim

3 Logical Architecture & Design (7+2 minutes)

Clock	16:20:00 – 16:29:00 (7 min talk + 2 min Q&A; talk ends 16:27:00)
Owner	Wang Ziang (Eric)
Slides	7, 8, 9, 10, 11

Guideline requirement (V2.8)

3a — Highlight the key architectural decisions that influence the logical architecture.
3b — Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views.
3c — Where relevant, highlight the elements of Domain Driven Design that help to shape the design of the solution as a platform. In particular, describe how the platform is designed to interact with the producers and consumers.

Tip

Slide 7 is the most important slide of the talk: it is the one decision assessors will remember. Rehearse it until it fits in 90 seconds without rushing.

Slide 7 Why three services, not five

16:20:00–16:21:30 · 90 s

Speaker Eric · covers guideline item **3a**

Say

I'm Eric. Our first decision was where to split. The obvious answer is one service per business area: catalog, orders, payments and so on. But when a payment comes in from Stripe, one database transaction confirms the payment, creates the order, takes the stock and adds the buyer to the team. You can see it on the left: four steps, one commit. If any step fails, everything rolls back, and that is exactly what protects a buyer's money. Splitting that would need sagas and compensation everywhere. We estimated five services at 50 to 55 person-days, our whole budget, so we rejected it. Our rule became simple: only split where no transaction crosses. Identity is used through one small interface, and nothing calls notifications at all. That gives us two reusable platform services and one transactional core.

≈ 136 words · ≈ 63 s at 130 wpm · slot 90 s

Cues

- Pause on the left box: this is the one decision assessors should remember.

Hand-over *"A few more decisions follow from that."*

Slide 8 Four decisions shape the logical design

16:21:30–16:22:50 · 80 s

Speaker Eric · covers guideline item **3a**

Say *From that first decision, three more follow, and each one solves a problem the split creates. First, services talk through events, and the event is written in the same transaction as the change. So an event is only sent if the change actually happened; we never tell anyone about a payment that rolled back. Second, all services share one login session, so any service can check your role without calling identity on every request. Third, when commerce can't reach identity, it refuses the purchase instead of guessing. A purchase is either fully done or not done at all, never half-paid. The cost is a small delay on notifications, about two seconds, and purchases pausing during an identity outage, which we'll show you live.*

≈ 123 words · ≈ 57 s at 130 wpm · slot 80 s

Hand-over *"Here's how the pieces fit together."*

Slide 9 Logical view: one direct call, everything else by events

16:22:50–16:24:20 · 90 s

Speaker Eric · covers guideline item **3b**

Say *This is our logical deployment view: two platform services at the top, the business application below, the event bus between them. Requests come in through CloudFront and a load balancer, which routes them to the right service by URL. There's only one direct call between services: commerce asks identity whether a user is active. That red arrow is the only place one service waits for another, so it's also the only place we had to design a failure mode for. Everything else goes through events. Commerce writes an event into an outbox table, a relay publishes it to an SNS topic, and each consumer gets its own queue. Today that's notifications and an audit log. Adding the audit consumer took a few lines of configuration and no change to commerce: producers don't need to know who's listening. Each service has its own database, and they all share one session store.*

≈ 150 words · ≈ 69 s at 130 wpm · slot 90 s

Cues

- `[VERIFY: "a few lines of configuration": find the audit-consumer PR and count lines (open item 1). If it cannot be found, say only "without any change to commerce"]`
- The simpler box diagram is backup B13 if a plainer companion is needed in Q&A.

Hand-over *"These boundaries come from the domain model."*

Slide 10 Bounded contexts decide the services; events are the contract 16:24:20–16:25:35 · 75 s

Speaker Eric · covers guideline item **3c**

Say *We found these boundaries by walking through the life of a team: created, joined, paid, succeeded or expired, refunded. That gave us six bounded contexts. Four of them, catalog, group buying, orders and payments, share transactions, so they live together in commerce. Identity and notification each stand alone. Inside commerce, contexts talk through domain events like TeamExpired. When an event leaves the service, it becomes a versioned integration event, for example TeamExpired version 1. That's a public contract, and it only carries IDs, never emails, so personal data stays inside identity. It also means a new consumer, like the audit log, can use our events without depending on our internal model.*

≈ 111 words · ≈ 51 s at 130 wpm · slot 75 s

Cues

- `[CONFIRM: Option A (as built) or Option B (context map on the right half) — Eric decides by Tuesday; B only if it fits 75 s (open item 14)]`
- The context map is backup B3 either way.

Hand-over *"So how do the two sides use the platform?"*

Slide 11 How producers and consumers use the platform, and what reuse costs 16:25:35–16:27:00 · 85 s

Speaker Eric · covers guideline item **3c**

Say *On the left are the producers, on the right the consumers, and the platform in the middle serves both. Sellers and admins publish listings; buyers start, share, join and track teams. Both sides use the same identity. Anything that happens in commerce becomes an event, and a new consumer just subscribes with its own queue. That's how the platform grows: we added the audit consumer without touching commerce, and an analytics or email consumer would work the same way. Reuse is measurable. Adding the audit consumer took* `[FILL 19 Oct: N]` *lines and zero changes to commerce. Adding a whole service is one entry in our Terraform services map, which creates its task, route, scaling and logs. And every client talks to one published API contract, from which the web client is generated.*

≈ 130 words · ≈ 60 s at 130 wpm · slot 85 s · if short of time: If over time, drop "That's how the platform grows...same way." (the measured line says it with a number).

Cues

- If the audit PR count is not found by Tuesday, the row is dropped from the slide: then also drop the sentence "Adding the audit consumer took N lines...".
- 16:27:00 — questions (2 minutes).
- 16:28:00 — Yorik starts the k6 load test (muted). Lim confirms in the backchannel.

Hand-over (after questions) "Liu Jin will show how this runs on AWS."

Q&A (16:27–16:29)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner

Isn't commerce-svc still a monolith?	Deliberately: one webhook transaction spans payment, order, catalog and groupbuy. Splitting it needs a saga with stock compensation, 50–55 person-days. It stays COLA-modular inside (backup B5); payment extraction (outbox + saga) is the roadmap (AISS-02, backup B2).	Eric
Why not one service per bounded context?	Same reason; point back to the one-transaction evidence and the rejected five-service option on slide 7. Context map: backup B3.	Eric
How does authorisation work across services?	Identity writes roles and permissions into the Sa-Token session in Valkey at login; other services read the session, with no per-request call. A role change applies at next login.	Eric
Why version integration events?	They are a public contract between independently deployed services; a consumer ignores unknown versions safely, so a producer can add V2 without a lock-step release.	Eric
What does DDD actually change in your design?	Bounded contexts gave the candidate boundaries; aggregates and transactions decided which ones could become network boundaries; domain vs integration events separate internal change from public contracts. Aggregates and state machines: backup B4.	Eric
How do outside producers and consumers get API access?	One committed OpenAPI contract with a generated client, checked on every PR; versioned events; /internal/** not routed. Not built: API gateway, per-client keys, rate limiting (AISS-04). Backup B10.	Eric

4 Physical Architecture & Design (7+2 minutes)

Clock	16:29:00 – 16:38:00 (7 min talk + 2 min Q&A; talk ends 16:36:00)
Owner	Liu Jin (4a–4c), Yu Qianshuo (Yorik) (4d)
Slides	12, 13, 14, 15, 16, 17

Guideline requirement (V2.8)

4a — Highlight the key architectural decisions that influence the physical architecture.
4b — Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution.
4c — Describe the persistence design of the solution.
4d — Describe the detailed design of an architecturally representative user case or user story.

Tip

Liu Jin owns slides 12–15 (about 4.5 minutes); Yorik owns 16–17 (about 2.7 minutes). The hand-over line is on slide 15. Yorik unmutes during the last sentence. Slide 16 is the UML version only if Yorik gives the go after rehearsing it; otherwise present B15 and talk through its numbered steps.

Slide 12 Physical decisions: managed, disposable, low cost

16:29:00–16:30:25 · 85 s

Speaker Liu Jin · covers guideline item **4a**

Say

I'm Liu Jin. We run containers on Fargate with ARM processors: no servers or Kubernetes cluster to manage, native auto-scaling, and about 20 percent cheaper. Kubernetes would cost more than all our compute just for its control plane. The whole environment is created when we need it and destroyed afterwards, so it costs about 20 cents an hour. We use one database server with a separate database per service, and a small NAT instance instead of a NAT gateway, about 4 dollars instead of 43 a month. The trade-off is less redundancy, which is acceptable for this project. And there are no long-lived cloud keys anywhere. And this is measured. Across seven sessions on 15 to 17 October, the runtime's database, cache and containers cost forty-nine cents in total.

≈ 129 words · ≈ 60 s at 130 wpm · slot 85 s

Hand-over *"Here's the resulting deployment."*

Slide 13 Physical view: private services behind CloudFront, across two zones

16:30:25–16:31:50 · 85 s

Speaker Liu Jin · covers guideline item **4b**

Say

This is the deployment in Singapore: one network across two availability zones. Only the load balancer and the NAT sit in public subnets; the services, database and cache are private. CloudFront is the only way in. It serves the web app from S3 and forwards API calls to the load balancer, which rejects anything without CloudFront's secret header. Images live in ECR, secrets in Parameter Store, events in SNS and SQS, and everything is monitored in CloudWatch. Only CloudFront, S3, ECR and the deploy roles are permanent; everything in the dashed box is rebuilt on every release. So security is layered: one public entry point, private services, no stored keys, and secrets that never touch the code.

≈ 117 words · ≈ 54 s at 130 wpm · slot 85 s

Cues

- The diagram labels the two boundaries "permanent" (teal) and "rebuilt every release" (orange): point at them on "everything in the dashed box".
- Eric checks the dashboard backstage: commerce task count should start rising around 16:35.
- The simpler diagram is backup B14.

Hand-over *"Each service scales on its own signal."*

Slide 14 Each service scales on its own signal, and how far it goes

16:31:50–16:32:35 · 45 s

Speaker Liu Jin · covers guideline item **4b**

Say

Commerce scales on requests per task, from two to six; notification on queue backlog, from one to four. One task handled 200 requests a second without saturating. On 19 October, under load, commerce scaled to `[FILL 19 Oct: N]` *tasks and served* `[FILL 19 Oct: X]` *requests a second at* `[FILL 19 Oct: Y]` *milliseconds. At six tasks we project about 1,200 for browsing; beyond that the next limit is the single database.*

≈ 64 words · ≈ 30 s at 130 wpm · slot 45 s · if short of time: If over time, skip the chart narration; read only the capacity line.

Cues

- Never present the projection as a measurement. If V3 did not run, say "the three-service run is not yet measured" and keep only the 16 Oct chart.

Hand-over *"Now the data."*

Slide 15 Each service owns its data

16:32:35–16:33:20 · 45 s

Speaker Liu Jin · covers guideline item **4c**

Say *Each service owns its own database on the shared server, and database permissions stop a service from reading anyone else's tables. The cache only holds sessions, never business data. On sensitive data: personal details stay in identity, we never store card data because Stripe handles payments, and events carry only IDs. Two buyers can race for the last seat, so we always lock in the same order, team, then purchase intent, then payment, which avoids deadlocks and decides the race. And scheduled jobs take a lease in the database, so with six commerce tasks only one runs them. Yorik will now walk through one use case in detail.*

≈ 108 words · ≈ 50 s at 130 wpm · slot 45 s · **over by 5 s: rehearse at pace or trim — If over time, cut "On sensitive data: ... only IDs." (it is on the slide).**

Hand-over *"Yorik, over to you."*

Slide 16 Team expiry, end to end: one transaction in, one transaction out

16:33:20–16:34:55 · 95 s

Speaker Yorik · covers guideline item **4d**

Say *This is the team expiry path. Every minute, a scheduled job holds a database lease, so only one commerce task runs it.* **[click]** *In one transaction it locks an overdue team, expires it, cancels the orders, records the refund work and writes a TeamExpired event to the outbox. Either all of that commits or none of it does.* **[click]** *After commit, the relay publishes the event every two seconds to SNS, which routes it to the notification queue. The notification service receives it, checks the event ID so a duplicate does nothing, and creates a notification for* `[VERIFY: each member / the team owner]` *in one transaction. Only then is the message deleted.* **[click]** *Refunds run separately: a refund worker claims the next piece of refund work every thirty seconds and calls Stripe with an idempotency key, so a retry can never refund twice. If Stripe fails, the work stays and is retried. So the expiry is one transaction in commerce and one in notification, with the queue between them, and the refund is retried safely on its own.*

≈ 170 words · ≈ 78 s at 130 wpm · slot 95 s

Cues

- Three clicks: 1 the expiry transaction loop; 2 relay → SNS → SQS → notification (steps 2–8); 3 the refund worker loop.
- `[CONFIRM: go/no-go on the UML swap after Yorik rehearses it (open item 13). No-go: present backup B15 (numbered-step diagram) and talk through its steps]`
- `[VERIFY: step 8 recipient (owner or each member) in the notification-svc consumer code (open item 2); fix the slide and this line together]`

Hand-over *"And when things go wrong?"*

Slide 17 Failures are expected, and handled

16:34:55–16:36:00 · 65 s

Speaker Yorik · covers guideline item **4d**

Say *Queues deliver at least once, so we planned for failure. A duplicate is ignored because its ID is already recorded. A message that keeps failing is moved to a dead-letter queue after five tries and an alarm fires, while everything else keeps flowing. We tested this locally: a broken message was received five times, then moved aside. If publishing fails, the event simply stays in the outbox and is retried. So: no lost events, no double notifications, and bad messages are isolated. You'll see the dead-letter queue in the demo.*

≈ 90 words · ≈ 42 s at 130 wpm · slot 65 s

Cues

- `[FILL 19 Oct: if V7 ran on AWS on 19 Oct, replace "We tested this locally" with the AWS result]`
- 16:36:00 — questions (2 minutes). Liu Jin answers infrastructure and data; Yorik answers events and failures.

Hand-over (after questions) "Pan will show how we build and ship it."

Q&A (16:36–16:38)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
What is your bottleneck at higher scale?	The single RDS instance and the single active outbox relay (`[VERIFY: "about 50 events/s" — measured or estimated? Say which (open item 5)]`). Paths: separate instances or read replicas; relay sharding by aggregate ID or CDC.	Liu Jin
Single RDS and single-AZ Valkey — single points of failure?	Yes, a cost decision for a demo; ECS tasks span two AZs. Multi-AZ RDS and a Valkey replica are the production change (AISS-01, backup B2).	Liu Jin
Why Fargate over EKS?	No control-plane fee (about US$73/month), native service auto scaling, no node management; three services do not need Kubernetes. Our options study compared both (AD-07, AD-13, backup B1b).	Liu Jin
How are /internal endpoints protected?	Not routed by the ALB, admitted only from peer security groups, absent from the public contract (InternalPathsContractTest). No service token — an accepted limitation (AISS-04), cut on 11 Oct as low value for the demo.	Liu Jin
How did you pick 12,000 requests per target per minute?	V2: one task held 200 RPS at p95 ≤ 57 ms and zero errors on 16 Oct without saturating; 200 × 60 = 12,000. It is a tested lower bound, validated on the distributed shape by V3.	Liu Jin
How do you avoid losing or duplicating events?	Outbox row in the business transaction (no dual write); at-least-once delivery; the consumer records (eventId, consumer) with its effect in one transaction. SQS MessageId is never the key, because retries create new ones.	Yorik
What happens when identity is down?	Browsing continues; purchases and admin writes fail closed with 500; a Stripe webhook rolls back and Stripe redelivers. Shown live in the demo. Limitation: checkout depends on identity at run time (AISS-03).	Yorik
Why SNS + SQS and not Kafka, EventBridge or RabbitMQ?	Managed, pay-per-request, no idle cost; SNS fan-out with filter policies; SQS backlog drives consumer scaling and gives DLQs. EventBridge is neither cheaper nor simpler for one producer; Kafka is overkill at this volume.	Yorik

Show me the payment path, not expiry.	Backup B6: UC4 sequence (signature check outside the transaction; one transaction for dedupe, lock order, admit, order, stock, outbox).	Yorik

5 DevOps and Development Lifecycle (6+1 minutes)

Clock	16:38:00 – 16:45:00 (6 min talk + 1 min Q&A; talk ends 16:44:00)
Owner	Panpakorn Siripanich (Pan)
Slides	18, 19, 20, 21, 22

Guideline requirement (V2.8)

5a — Describe the strategy to manage the source code and artifacts.
5b — Describe the DevOps pipeline and procedures.
5c — Highlight any technical findings and issues encountered.

Tip

Segment 5 now has five slides in the same 6 minutes: 18 (65 s) + 19 (70 s) + 20 (95 s) + 21 (75 s) + 22 (55 s) = 360 s. The hand-over to the demo moved from slide 21 to slide 22.

Slide 18 One main branch; production changes only by promotion

16:38:00–16:39:05 · 65 s

Speaker Pan · covers guideline item **5a**

Say

I'm Pan. All work happens on short feature branches that merge into one main branch, dev, after review and automated checks. Production is a separate branch, prod, and the only way to change it is a promotion pull request from dev; nobody can push to it directly. For artefacts, we store container images by their digest, not by tag, record which digests are the current release, and keep one API contract file in the repository.

≈ 75 words · ≈ 35 s at 130 wpm · slot 65 s

Hand-over *"Every change is checked before it merges."*

Slide 19 Every pull request is checked before it can merge

16:39:05–16:40:15 · 70 s

Speaker Pan · covers guideline item **5b**

Say

Every pull request runs four groups of checks. Build and test, including an API contract check and 80 percent test coverage on new code. Container builds: we build every service image and actually boot it. Security: secret scanning, dependency and image scanning, and static analysis. And infrastructure: Terraform checks, tests and policy checks. Over the month we ran over five thousand of these, about ninety-five percent green, in under nine minutes. The container check matters most for us: it catches a change that compiles fine but breaks one of the three services at start-up. If any check fails, the merge is blocked.

≈ 102 words · ≈ 47 s at 130 wpm · slot 70 s · if short of time: If over time, one sentence for all four groups.

Cues

- `[VERIFY: boot check and CodeQL exist in .github/workflows/ (open item 4). If not, drop "and actually boot it" / "static analysis" from both slide and script]`
- 16:40:00 — expiry-burst decision (Plan A or Plan B, see the demo run book). Liu Jin acts in the backchannel only.

Hand-over *"And this is how a change reaches production."*

Slide 20 One promotion builds, deploys, tests and releases, then destroys it all

16:40:15–16:41:50 · 95 s

Speaker Pan · covers guideline item **5b**

Say

This is the key part. Merging a promotion pull request is the only thing that deploys. The pipeline builds the images, creates the entire AWS environment from scratch, deploys the services and runs smoke tests through CloudFront. The services sit behind a circuit breaker, so a release that fails its health checks rolls back by itself. Then the pipeline marks those images as the release, destroys everything and checks that nothing was left behind. That last step always runs, even if something failed earlier. Because every release builds the environment from scratch, every release also doubles as a disaster-recovery test. Our first run on 6 October, with one service, took 28 minutes. The three-service shape takes 51: creating the environment is about half an hour, and fifteen minutes of that is waiting for new SNS subscriptions to start routing, which brings us to what we learned.

≈ 146 words · ≈ 67 s at 130 wpm · slot 95 s

Cues

- `[VERIFY: build, release and gap durations from run 39008208244 (open item 6); only 32 min, 709 s and 51 min are measured]`

Hand-over *"Three lessons we learned on the way."*

Slide 21 What the real cloud taught us

16:41:50–16:43:05 · 75 s

Speaker Pan · covers guideline item **5c**

Say *Three lessons. First, our Terraform tests passed with mocks, but the first real deployment failed on missing permissions: mocks only prove what you told them, so we now test permissions for real. Second, new SNS subscriptions can take up to fifteen minutes before they route messages; nothing errors, messages just quietly go nowhere, so our deploy now waits until routing is confirmed. Third, a start-up that was interrupted and retried reported success but left auto-scaling paused. Deployed doesn't mean scaling, so now we check.*

≈ 84 words · ≈ 39 s at 130 wpm · slot 75 s

Cues

- The full findings table is backup B8.

Hand-over *"And who and what can touch AWS."*

Slide 22 No standing keys: who and what can touch AWS

16:43:05–16:44:00 · 55 s

Speaker Pan · covers guideline item **5b**

Say *Security starts above the account. Our project account sits in an AWS Organization whose policies pin us to Singapore and block expensive services, and those policies bind even our own administrators. People sign in through Identity Center with one-hour sessions. The pipeline uses GitHub's OIDC to get short-lived credentials, with separate roles to plan, deploy and apply, so there are no AWS keys in GitHub. One exception, disclosed: two IAM users for database port forwarding, still bounded by the same guardrails. Combined with private subnets and the CloudFront-only entry you saw earlier, that is our security model. Eric will now show you all of this live.*

≈ 106 words · ≈ 49 s at 130 wpm · slot 55 s

Cues

- 16:44:00 — questions (1 minute).
- 16:44:30 — Lim stops sharing; Eric starts sharing the demo window.

Hand-over (after questions) "Eric, over to you."

Watch out

At 16:44:30 Lim stops sharing and Eric starts sharing the demo window, even if a question is still being answered. The answer continues over Eric's screen.

Q&A (16:44–16:45)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How do you stop a change from breaking one of the services?	Every pull request builds and boots every service image, and architecture tests reject calls that cross service boundaries (18 ArchUnit rules, backup B5).	Pan
Why destroy the environment after every promotion?	Cost (about US$0.20/hour versus about US$125/month always-on; measured US$0.49 for 7 sessions) and proof of reproducibility: every release is a rebuild from zero, with teardown verified by env-status. Backup B9.	Pan

What security controls are in the pipeline?	gitleaks, Trivy (dependencies, images, IaC config), `[VERIFY: CodeQL]`, checkov, SonarCloud gate; OIDC roles scoped to prod; secrets only in SSM; origin-secret header on the ALB. Account-level controls on slide 22.	Pan
How do you roll back?	The ECS deployment circuit breaker rolls back a failing service automatically; release digests in SSM are the last-known-good set that env-up deploys.	Pan
Why do IAM users still exist?	Two users for SSM port forwarding (one active key), disclosed on slide 22 and bounded by the SCPs; everything else is Identity Center or OIDC.	Pan → Lim

6 Demo (8+2 minutes)

Clock	16:45:00 – 16:55:00 (8 min talk + 2 min Q&A; talk ends 16:53:00)
Owner	Eric and Yorik (drive), Pan (pipeline)
Slides	23, 24

Guideline requirement (V2.8)

6a — Demonstrate some of the significant use cases of the solution.
6b — Demonstrate how the solution deals with some of the important quality attributes e.g., scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).

How the demo meets 6a and 6b

Step	Shows	Quality attribute (guideline 6b wording)	Evidence on screen
1	Buy as a customer, across three services	Functionality	All four views load; at least one notification is listed.
2	Auto-scaling under a load test	Scalability, performance	Commerce running tasks above 2 (about 3–4 at 600 requests a second); k6 error rate below 1%.
3	Stop a server, nothing breaks	Availability	Replacement running within about 2 minutes; no visible error spike.
4	Identity goes down, payments stay safe	Consistency, security	Browse works; admin change returns 500; identity back to 1 task before 16:52:30.
5	Event backlog and the dead-letter queue	Scalability, reliability	Dead-letter queue shows about 1 message; alarm in ALARM; main queue back to 0.
6	The pipeline that deployed it	Maintainability	Checks green; run green including destroy; artifact present.

Scalability is shown twice (HTTP auto-scaling in step 2 and queue scaling in step 5), performance through the scripted k6 test in step 2, and maintainability through the pipeline in step 6, as the guideline asks. Steps 3–4 add availability and consistency, which are central to the decisions on slide 8.

Principle: start everything slow before the demo

The 8 minutes only **show results** plus two fast live actions (task stop, identity outage). The k6 V3 run starts at 16:28 and follows the reviewed ramp: 20% → 100% of the 200 RPS baseline over 5 minutes (16:28–16:33), up to 300% = 600 RPS over 10 minutes (16:33–16:43), hold 10 minutes (16:43–16:53), ramp down 5 minutes. At 600 RPS and a 200 RPS-per-task target, commerce should settle at 3–4 tasks, so scale-out events fall between about 16:35 and 16:45 — before step 2.

Decision: Plan A or Plan B for the expiry burst (decide at the 20 Oct rehearsal)

	Plan A — default	Plan B — live burst
Poison message (V7)	Injected at 15:30; in the DLQ with the alarm in ALARM by about 15:40	Injected at about 16:41, right after the V4 seed is confirmed; reaches the DLQ after 5 receives × 60 s visibility, about 16:47; alarm by about 16:49
Queue scaling (V4)	Shown from the 19 Oct V4 recording and its dashboard capture	Live: Liu Jin runs `v4-burst.sh` at 16:40 (2,000 teams due 60 s later)
Preconditions	None beyond the normal session	A separately reviewed runtime apply with `v4_demo_enabled=true` before 12:00 (`env-up` does not set it); one stable notification task; quiet main queue and DLQ at 16:40; new run label
Risk	Low	Tight timing; backlog may drain before scaling reacts (SCALING.md warns of this); fallback is the same recording as Plan A

Watch out

Why the 14 Oct guideline timing does not work: `v4-burst.sh` refuses to seed unless the notification queue **and the DLQ** are quiet, so a poison message injected at 15:30 blocks a 16:40 burst. Without `v4_demo_enabled=true` the expiry sweep handles 100 teams a minute, which drains without any scale-out. Liu Jin posts `PLAN A` or `PLAN B` in the backchannel at 16:40.

Demo preconditions (checked at 15:30 by Liu Jin and Eric)

- Environment up from release digests, smoke green, health `UP`; commerce 2/2, identity 1/1, notification 1/1 running.
- Demo admin granted and signed in again; startable listings seeded (A.5).
- Scaling targets **not suspended**: commerce 2–6, notification 1–4 (read back; see finding 3 on slide 21).
- Plan A: DLQ shows 1 message, alarm `blocbuy-swe5001-notification-dlq-not-empty` in ALARM. Plan B: main queue and DLQ empty.
- Notifications JSON for the demo buyer shows at least one entry (Plan A: from the morning smoke or a seeded expiry).
- Load machine: Docker running, k6 image pulled, `BASE_URL`, digest and run label exported (A.7); a dry `http-preflight.sh` passes.
- Fallback clips from 19 Oct open in a player on Liu Jin's machine, one per step, tested with Zoom “share sound” off. `[FILL 19 Oct: list which steps have no 19 Oct recording; those use Plan C]`

Screen layout (Eric's demo window)

One browser window, tabs left to right, plus one terminal. Share the window, not the desktop; notifications off; browser zoom 125%.

Tab	Content	Prepared state
1	HTML deck opened at slide 23 (`swe5001-presentation.html#23`)	Also used for slide 24 at the end; backups B1–B15 one key away (B)

2	SPA `https://d3lvq02o2e7tcn.cloudfront.net`	Product list loaded; signed in as the demo admin (Google)
3	Admin product list in the SPA	Admin listing visible; one harmless admin write prepared for step 4
4	`/api/v1/notifications` for the demo buyer	JSON list with at least one notification
5	CloudWatch dashboard `blocbuy-swe5001`	1-hour window, auto-refresh 10 s; Edge, Services, Messaging rows visible
6	ECS console: cluster `blocbuy-swe5001`, service `commerce-svc` → Tasks	Filtered to running and stopped tasks
7	SQS `blocbuy-swe5001-notification-dlq`; CloudWatch alarm `blocbuy-swe5001-notification-dlq-not-empty`	Plan A: 1 message, ALARM
8	GitHub: 19 Oct promotion PR `dev` → `prod` and its `swe5001-deploy` run	Checks tab and run graph open; never the Merge button
T	Terminal mirroring k6 on the load machine	Live summary: request rate, p95, error rate

Opening (slide 23)

Slide 23 Live demo 16:45:00–16:45:15 · 15 s

Speaker Eric · covers guideline item **6a**

Say *In the next eight minutes we'll show the buying journey across three services, auto-scaling and failure handling under a load test that has been running since 16:28, and the pipeline that deployed it all.*

≈ 34 words · ≈ 16 s at 130 wpm · slot 15 s · **over by 1 s: rehearse at pace or trim**

Cues

- Eric's tab 1 shows this slide (swe5001-presentation.html#23). Follow the demo run book step by step.
- `[CONFIRM: footer after 19 Oct — keep "everything you see is live on AWS" only if every step is live; otherwise "Live on AWS; recorded steps are marked"]`

Hand-over *"First, the use cases."*

Step by step

Step 1 Buy as a customer, across three services

16:45:15 – 16:46:15

Proves Functionality · guideline **6a** · Eric

Operate

1. Tab 2: product list with team prices and countdowns (commerce).
2. Point at the signed-in user in the header (identity, Google sign-in).
3. Tab 3: admin product list (commerce reads the role from the shared session).
4. Tab 4: refresh the buyer's notifications (notification service).

Say *This is the buyer view: team prices and live countdowns, served by commerce. I'm signed in with Google through identity. This admin page works because commerce reads my role from the shared session, without calling identity. And these notifications were written by the notification service from events it received. Three services, one session.*

Expect All four views load; at least one notification is listed.

Fallback Screenshots from 19 Oct (fallback/1-use-cases). If the live view is not as expected within 20 seconds, Eric posts `CLIP 1` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 2 Auto-scaling under a load test

16:46:15 – 16:47:45

Proves Scalability, performance · guideline **6b** · Eric (Yorik mirrors k6)

Operate

1. Tab 5: CloudWatch dashboard, Services row: commerce running tasks.
2. Edge row: request count and p95 response time.
3. Terminal: k6 live summary (requests per second, p95, error rate).

Say *Since 16:28 a k6 script has been ramping up browse traffic to three times our measured baseline, 600 requests a second. Commerce has scaled from two tasks to the number you see here. Response times rose while new tasks started and came back down once they joined, and the error rate stayed under one percent. (Read the task count, p95 and error rate off the screen.)*

Expect Commerce running tasks above 2 (about 3–4 at 600 requests a second); k6 error rate below 1%.

Fallback Recording of the 19 Oct load test (fallback/2-v3.mp4). If the live view is not as expected within 20 seconds, Eric posts `CLIP 2` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 3 Stop a server, nothing breaks

16:47:45 – 16:48:30

Proves Availability · guideline **6b** · Yorik operates; Eric narrates

Operate

1. Yorik runs command A.3 (stop one commerce task).
2. Tab 6: ECS console, commerce tasks: one goes STOPPED, a replacement starts.
3. Terminal: k6 error counter.

Say *Yorik has just stopped one commerce task in the middle of the load test. ECS has already started a replacement, and the load balancer routes around the missing one, so the error rate doesn't move. We'll check the replacement at the end.*

Expect Replacement running within about 2 minutes; no visible error spike.

Fallback Recording of the 19 Oct drill (fallback/3-v5.mp4). If the live view is not as expected within 20 seconds, Eric posts `CLIP 3` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 4 Identity goes down, payments stay safe

16:48:30 – 16:50:00

Proves Consistency, security · guideline **6b** · Yorik operates; Eric narrates

Operate

1. Yorik runs command A.4 (identity desired count 0).
2. Tab 2: browsing still works.
3. Tab 3: an admin change now fails with a server error, not 'user inactive'.
4. Yorik restores identity to 1 immediately.

Say *Now identity is down. Browsing still works, because it only needs commerce. But anything that needs to know the user is active now fails safely with a server error, not with "user inactive". That difference matters: for a Stripe payment, a server error rolls everything back and Stripe retries later, so no paying buyer is wrongly refunded. Yorik is bringing identity back now.*

Expect Browse works; admin change returns 500; identity back to 1 task before 16:52:30.

Fallback Recording of the 19 Oct identity drill (fallback/4-identity.mp4). If the live view is not as expected within 20 seconds, Eric posts `CLIP 4` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 5 Event backlog and the dead-letter queue

16:50:00 – 16:51:30

Proves Scalability, reliability · guideline **6b** · Eric

Operate

1. Tab 5, Messaging row: published events, notification queue backlog; Services row: notification tasks.
2. Tab 7: dead-letter queue with 1 message; its alarm in ALARM.

Say *Plan A: Here's our 19 October run: 2,000 teams expired at once, the queue backed up, notification scaled out and drained it to zero. And this is live: the dead-letter queue holds one broken message we sent this afternoon. It was tried five times, moved aside, and the alarm fired, while every other message kept flowing. Plan B: At 16:40 we expired 2,000 teams. You can see the spike, the backlog, notification scaling out and the queue draining.*

Expect Dead-letter queue shows about 1 message; alarm in ALARM; main queue back to 0.

Fallback Recording of the 19 Oct runs (fallback/5-v4-v7.mp4). If the live view is not as expected within 20 seconds, Eric posts `CLIP 5` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 6 The pipeline that deployed it

16:51:30 – 16:52:30

Proves Maintainability · guideline **6b** · Pan narrates; Eric clicks

Operate

1. Tab 8: the 19 Oct promotion pull request: required checks and approval.
2. Open its deploy run: build → create → release → destroy → evidence, with durations.
3. Open the evidence artifact list. Never click Merge.

Say *This is the promotion that put today's code on AWS: three required checks and one approval. The run built the images, created the environment from scratch, tested it, released it and destroyed it, and here's the evidence it saved. This morning's environment was created from exactly these release images. (Read the total run time off the screen.)*

Expect Checks green; run green including destroy; artifact present.

Fallback Screenshots of the pull request and run (fallback/6-pipeline). If the live view is not as expected within 20 seconds, Eric posts `CLIP 6` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Step 7 Close the loop 16:52:30 – 16:53:00

Proves Availability · guideline **6b** · Eric, then Lim

Operate
1. Tab 6: commerce back to full strength; identity 1/1 running.
2. Tab 1: thank-you slide (slide 24).

Say *The replacement commerce task is healthy and identity is back. That's our demo.*

Expect All services at their desired count.

Fallback If identity isn't back yet, say so in one sentence and move on. If the live view is not as expected within 20 seconds, Eric posts `CLIP 7` and says: *"Here is the same step from our 19 October run."* If there is no 19 Oct recording for this step, use Plan C (§7).

Watch out

Never live: clicking Merge on any PR into `prod` (a promotion destroys this environment); opening Parameter Store values, Terraform state or plan output; scaling commerce or notification by hand (autoscaling owns them); running `env-up.sh` while k6 is running. The identity outage must be restored before step 6; if identity is not back by 16:52:30, say so in one sentence and move on.

Tip

Narrate results, not clicks: "task count went from two to four", not "now I'm clicking on the services row". No troubleshooting on air.

Closing slide

Slide 24 Thank you 16:53 · Q&A

Speaker Lim · covers guideline item **6b**

Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Cues
- 16:55:00 — end. Lim thanks the assessors. Liu Jin tears the environment down only after the meeting ends.
- Backup slides B1–B15 follow this slide (press → or B for the index). Never show them unasked.

Q&A (16:53–16:55)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How did you choose the scaling target?	From the V2 baseline (p95 < 500 ms, errors < 1% at every plateau up to 200 RPS), converted to 12,000 requests per target per minute; the 19 Oct load test checks it on the three-service shape.	Eric
Is the scale-out linear?	Answer only from the 19 Oct load-test analysis: throughput at one versus four tasks, or say plainly that it was not measured. Backup B12.	Eric
What analytics does the platform	Operational analytics in use (dashboard, alarms, the load results that set the scaling target). Business analytics (fill rate, time-to-fill, referrals, refund rate) are	Lim

support?	in the data but not built; next step is a read-only analytics consumer on the event bus. Backup B11.	
Can you show … again?	The demo screen stays up; Eric drives, Yorik for drills, Pan for the pipeline.	Lim routes
Any question left over from earlier segments	Answered by that segment's owner, 30–45 s each.	Lim routes

7 Contingencies and time recovery

If something fails

Situation	Detected by	Response and words	Who
Morning up fails	Workflow red	Read the cause, run down, retry once; deadline 12:00. If it fails again, present the demo entirely from recordings and say so on slide 5.	Liu Jin
Presenter's audio or connection drops	Silence > 5 s	Backup continues from the slide card: Lim ↔ Pan, Eric ↔ Yorik, Liu Jin ↔ Yorik.	Backup
Lim cannot share	Black screen	Eric shares the HTML deck from tab 1 for segments 1–5 (PDF as a last resort).	Eric
k6 does not start at 16:28	No `K6 UP` by 16:30	Yorik retries once before 16:31; otherwise step 2 uses the V3 recording.	Yorik
Commerce has not scaled by 16:46	Dashboard at 2 tasks	Show what is visible honestly ("the alarm needs three minutes of data"), then the V3 recording.	Eric
Stopped task not replaced	ECS tab	Show the ECS events list; then the V5 recording.	Eric
Identity outage step misbehaves	Admin write not 500	Restore identity immediately; show the identity-outage recording.	Yorik
DLQ empty or alarm OK	Tab 7	Show the V7 recording; mention the local redrive test (receive counts 1–5 then DLQ, 15 Oct).	Eric
GitHub slow or logged out	Tab 8	Show the pipeline screenshots.	Pan
Plan C (new): a step fails live and there is no 19 Oct recording for it	—	Show the local integration-test result for that step and say: *"verified locally, not yet on AWS."* Never present a local result as an AWS one.	Eric
Assessor asks for something not prepared	—	Offer to show it after the talk or in the report; do not improvise on the live environment. If a backup slide covers it, open it (B, then the row).	Lim

If the clock slips

Recover time in this order; never cut a guideline item, only depth.

Segment	Cut first	Saves
1	Slide 3: skip the seed sentence; slide 4: drop "and we added event-driven notifications"	15 s
2	Slide 6: name only three milestone dates	15 s
3	Slide 8: name the decisions, skip the cost sentence; slide 11: drop the "That's how the platform grows" sentence	35 s
4	Slide 12: skip the Kubernetes sentence; slide 14: skip the chart narration; slide 15: skip the sensitive-data sentence	45 s
5	Slide 19: one sentence for all four checks; slide 21: two findings instead of three; slide 22: skip the disclosed-exception sentence (it is on the slide)	50 s

6	Step 3 and step 4 merged into one narration while Yorik runs both	45 s

8 Timeline before, on and after the day

Before the day

Date	Do	Owner
Sun 18 Oct	Each owner reviews their slides and this script, and confirms their effort estimate on slide 6. Every marker has an owner in §9.	All; Lim collects
Mon 19 Oct	Sprint 4 promotion. Evidence session: V3, V4, V5, V7, identity outage, buyer E2E (#460), smoke on ECS. **Record every demo step** (screen + k6 terminal, 1080p, no secrets visible) into `fallback/1-use-cases` … `fallback/6-pipeline`. Capture the CloudWatch crops for slide 14 and B11. Save evidence under `docs/swe5001/evidence/`. `env-down` + clean `env-status`.	All; Liu Jin operates
Mon 19 Oct evening	Resolve every [FILL 19 Oct] from the evidence folder; move anything not run to "Still open" on slide 5 and to B12 as "not run on AWS; local result: …". Rebuild deck, PDF and this script.	Lim, with segment owners
Tue 20 Oct	**Full rehearsal** in Zoom, timed against the run sheet, with both share switches and the real 16:28 / 16:40 cues on the clock. **Decide Plan A or Plan B**, slide 10 Option A/B and the slide 16 go/no-go. Every speaker answers their Q&A rows in ≤ 45 s. No marker may remain after this rehearsal.	All; Lim times

Presentation day (Wed 21 Oct)

Time (SGT)	Do	Owner
≤ 10:30	A.1: nothing queued that could destroy the session. Plan B only: reviewed runtime apply with `v4_demo_enabled=true` first. A.2: dispatch `up` on `prod`; confirm SNS subscription emails during the 15-minute readiness wait; watch to green (15–25 min, ≈ 32 min for the distributed cold start).	Liu Jin
≤ 12:00	SPA loads, health `UP`, smoke green, scaling targets read back as not suspended. Sign in with the demo Google account, A.5 grant-admin, sign out and in, seed listings. If up failed: cause → `down` → retry once.	Liu Jin
13:30	Tech check in a test Zoom: mic, camera, share window, notifications off, wired network if possible. Load machine ready (A.7 exports, preflight).	All
15:30	Plan A: inject the poison message (A.6); confirm DLQ = 1 and alarm in ALARM by 15:45. Open demo tabs; run through §6 preconditions.	Liu Jin, Eric
16:00	Join the meeting as “Team 8 – Name”; backchannel open; fallback clips loaded.	All
16:15	Start; follow the run sheet.	Lim
16:28	`K6 GO` → Yorik starts V3 (A.7).	Yorik
16:40	`PLAN A` (nothing to do) or `PLAN B`: run A.8, then A.6 once the seed is confirmed.	Liu Jin
16:44:30	Share switch Lim → Eric.	Lim, Eric
16:55	End; thank the assessors.	Lim
≤ 17:30	Export dashboard screenshot, k6 summary, ECS events to the evidence folder. A.9: `down` and a clean `status`. Write every assessor remark into #461.	Liu Jin, Lim

After the presentation

- Teardown verified (A.9); record both run links in the day log. Evidence saved **before** teardown: log groups die with `runtime`.
- Every assessor remark goes into #461 the same evening; each becomes a change or a justified non-change in the report (due 16 Nov).
- Tag `swe5001-demo-20261021` on the `prod` commit that was shown (never a `v*` tag).

9 Open items before Tuesday's rehearsal

There are 7 `[FILL 19 Oct]`, 7 `[VERIFY]` and 4 `[CONFIRM]` markers across the deck and this script (counted in the script; the deck shows the same ones on the slides). They stay visible until resolved: **FILL** values only exist after the 19 Oct session, so never present a guess; **VERIFY** claims exist in one source but conflict with or are unconfirmed by another, so check the repo or run logs; **CONFIRM** needs team agreement.

#	Item	Owner	Blocks slide
1	Audit-consumer PR: line and file count	Yorik / Eric	9, 11
2	Expiry notification recipients: owner or each member	Eric	16
3	Scaling-restore defect: is a fix PR open?	Liu Jin	5
4	Boot check and CodeQL in CI: present in workflows?	Pan	19
5	Relay ceiling "~50 events/s": measured or estimated?	Yorik	14, Q&A
6	17 Oct run 39008208244 step durations	Liu Jin	20
7	Effort split agreed by all five	Lim	6
8	V3, V4, V5, V7, identity outage, buyer E2E results	All (19 Oct)	5, 14, 23, 24, B12
9	Q&A bank: AISS-02 → AISS-01 for single-zone data	Liu Jin	Q&A (done in this script)
10	UC7 referral data exists for the analytics answer	Lim	B11
11	Slide 22 and B10/B11 graphics in deck style	Lim	done; review
12	Edit and recolour the SVGs for slides 9, 13, 16	Lim	done; review
13	Yorik rehearses the UML expiry sequence; go/no-go on the slide 16 swap	Yorik	16
14	Slide 10 Option A or B	Eric	10
15	Regenerate HTML, PDF and script after 19 Oct	Lim	all

Effort per member `[CONFIRM: all five agree]`

Estimated from the planned hours of the issues each person delivered, credited to whoever actually did the work. Total ≈ 50 person-days.

Member	Person-days	Main area
Liu Jin	12.5	AWS stacks, pipeline, service split
Panpakorn (Pan)	11.5	Messaging, scaling, login, load tests
Yu Qianshuo (Yorik)	9	Event outbox and relay, failure drills

Lim Jiechao	8.5	Lead; AWS Organization, Identity Center and guardrails; architecture study and proof of concept; presentation
Wang Ziang (Eric)	8.5	Notification consumer chain, DLQ redrive, demo lead

A Appendix — operator commands

Run from an operator environment with the reviewed checkout (see the environment session runbook). Never paste outputs containing parameter values into chat or onto the shared screen.

A.1 Nothing queued that could destroy the session

```
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-env.yml --limit 5
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-deploy.yml --limit 3
```

A.2 Bring the environment up (release digests) and check health

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=up
curl -s https://d3lvq02o2e7tcn.cloudfront.net/api/actuator/health
aws application-autoscaling describe-scalable-targets --service-namespace ecs \
  --query 'ScalableTargets[].{id:ResourceId,min:MinCapacity,max:MaxCapacity,suspended:SuspendedStat
e}'
```

A.3 V5 self-healing — stop one commerce task under load

```
aws ecs list-tasks --cluster blocbuy-swe5001 --service-name commerce-svc
aws ecs stop-task --cluster blocbuy-swe5001 --task <task-arn> --reason "V5 drill"
aws ecs describe-services --cluster blocbuy-swe5001 --services commerce-svc \
  --query 'services[0].events[:5]'
```

A.4 Identity outage drill — stop and restore

```
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 0
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 1
```

A.5 Demo admin and startable listings

```
infra/swe5001/scripts/grant-admin.sh '<demo google email>'
BLOCBUY_BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net/api/v1 \
  BLOCBUY_TOKEN=<cookie value> scripts/seed-dev-demo-listings.sh
```

A.6 V7 poison message

Use the drill script delivered by #459 if it exists. Otherwise publish one malformed envelope to the topic with the routing attribute the notification subscription filters on (read the filter policy first; a message without it is filtered out and never reaches the queue):

```
TOPIC=$(aws sns list-topics --query "Topics[?ends_with(TopicArn, ':blocbuy-swe5001-domain-events')].
TopicArn" --output text)
aws sns list-subscriptions-by-topic --topic-arn "$TOPIC"     # find the notification subscription
aws sns get-subscription-attributes --subscription-arn <arn> --query 'Attributes.FilterPolicy'
aws sns publish --topic-arn "$TOPIC" --message '{"malformed":true}' \
  --message-attributes '{"eventType":{"DataType":"String","StringValue":"PaymentSucceeded"}}'
```

Add any further attribute the filter policy requires. With a 60 s visibility timeout and maxReceiveCount 5, expect the DLQ after about 5–6 minutes and the alarm a few minutes later.

A.7 V3 load (load machine, POSIX shell, from the repository root)

Ramp: 20% → 100% of KNEE_RPS in 5 min, to 300% in 10 min, hold 10 min, down 5 min (30 min total).

```
results=$(mktemp -d); chmod 700 "$results"
export BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net CONFIRM_TARGET=https://d3lvq02o2e7tcn.cloudfro
nt.net
export TEST_SHA=$(git rev-parse HEAD) IMAGE_SHA='<deployed commerce source sha>'
export EXPECTED_IMAGE_DIGEST='sha256:<deployed commerce digest>' RUN_LABEL=20261021-v3-presentation
docker run --rm --user "$(id -u):$(id -g)" \
  --mount "type=bind,source=$PWD/perf/k6,target=/scripts,readonly" \
  --mount "type=bind,source=$results,target=/results" \
  --env BASE_URL --env CONFIRM_TARGET --env TEST_SHA --env IMAGE_SHA --env EXPECTED_IMAGE_DIGEST --e
nv RUN_LABEL \
  grafana/k6:1.6.1@sha256:a5ad6bc089a08d77c3ec49f3db8c6fa7a148e4073efcac44c675dbaf3568d8e1 run \
  --env MODE=v3 --env TOPOLOGY=distributed --env KNEE_RPS=200 \
  --env PREFLIGHT_PATH=/results/preflight.json --env SUMMARY_PATH=/results/summary.json \
  --out json=/results/samples.jsonl /scripts/browse.js
```

A.8 V4 burst (Plan B only; requires the reviewed v4_demo_enabled=true deployment)

```
export V4_RUN_LABEL=v4-20261021-presentation V4_RECIPIENT_ID=<already-active synthetic identity uuid
>
export EXPECTED_IMAGE_DIGEST=sha256:<commerce digest> EXPECTED_NOTIFICATION_DIGEST=sha256:<notificat
ion digest>
export CONFIRM_V4_SEED=2000 V4_RESULTS_DIR=<new empty private directory outside the checkout>
bash infra/swe5001/scripts/v4-burst.sh
```

Restore `v4_demo_enabled=false` with a reviewed apply after the talk (or let `down` remove the runtime).

A.9 Tear down and verify

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=down
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=status
```

B Appendix — backup slides (Q&A only)

Backups follow slide 24 in the HTML deck and the PDF. In the deck press **B** for the index (BX) and click a row; slide 24 stays up otherwise. Each backup gives the AD/AISS IDs in the Q&A answers a slide to point to.

ID	Slide	Answers
B1a	Architecture decisions AD-01 to AD-06 (logical)	Why X over Y (logical)
B1b	Architecture decisions AD-07 to AD-14 (physical)	Why Fargate, NAT instance, one RDS, accounts
B2	Known limitations AISS-01 to AISS-06	Single points of failure, coarse commerce, /internal
B3	Context map	What DDD actually changes
B4	Aggregates and state machines	GroupDeal / Order lifecycle
B5	Layers and enforced boundaries	How boundaries are kept from eroding
B6	UC4: join a team by paying	The payment path in detail
B7	Technology and services	Exact services, sizes, versions
B8	Full findings list	What else went wrong
B9	Infrastructure as code and actual bill	Stacks, lifetimes, real cost
B10	API access for producers and consumers	Platform Engineering: robust API access
B11	Analytics: what the platform measures	Analytics success criterion
B12	Scale-out and resilience results	Evidence behind slides 5, 14, 17
B13	Logical view, simplified	Plainer companion to slide 9
B14	Physical view, simplified	Plainer companion to slide 13
B15	Team expiry, numbered steps	Plainer companion / fallback for slide 16